

Unidad 2: The any type

Se estudió el tipo especial `any`, que permite que una variable almacene cualquier tipo de dato. También se analizaron las desventajas de utilizar `any`, ya que desactiva la verificación de tipos de TypeScript y puede aumentar la probabilidad de errores en el código.

Resumen del capítulo

Last updated February 2024

Si quieres usar TypeScript en un proyecto nuevo, te recomendamos usar vitejs.dev.

En tu línea de comandos, ejecuta el siguiente comando:

```
npm create vite@latest
```

Luego ingresa el nombre del proyecto y elige **Vanilla** y después **TypeScript**.

Notarás que el `tsconfig.json` dentro de este nuevo proyecto contiene lo siguiente: `"include": ["src"]`. Esto le indica al compilador de TypeScript qué archivos debe procesar. `src` significa que procesará todos los archivos que comiencen con `src`, lo cual es equivalente a todos los archivos dentro de la carpeta `src`.

También puedes ajustar esto especificando una entrada `exclude`.

En el próximo capítulo, aprenderemos sobre TypeScript ESLint. ¡Nos vemos allí!

Resumen del capítulo

- TypeScript tiene un tipo especial llamado `any` que significa que la variable puede ser de cualquier tipo. Esto efectivamente *desactiva* la verificación de tipos para esa variable específica.
- Generalmente, debes evitar usar `any` ya que desactiva la verificación de tipos de TypeScript.
- Una variable con tipo `any` puede tener cualquier valor y TypeScript no podrá distinguir si va a ser una cadena, un número o cualquier otro tipo.
- `tsconfig.json` es un archivo de configuración que le indica al compilador de TypeScript cómo debe procesar tu código.
- Recomendamos habilitar la opción `strict` ya que te brinda muchos de los beneficios que ofrece TypeScript.

Unidad 3: TypeScript ESLint

Se aprendió a utilizar TypeScript ESLint para mejorar la calidad del código mediante reglas de análisis estático. Además, se revisó el uso del modo estricto (``strict``) y diferentes directivas para el manejo de errores y verificaciones de tipos.

Last updated November 2025

¡Gran trabajo!

💡 ¿Disfrutando el curso?

Considera compartir con tus amigos

Facebook

Twitter

Tumblr

E-Mail

Pinterest

LinkedIn

Reddit

XING

WhatsApp

Hacker News

VK

Telegram

Resumen del capítulo

Ahora que hemos aprendido sobre TypeScript ESLint, finalmente podemos empezar a aprender sobre los tipos primitivos restantes, cómo declarar el tipo de dato de las declaraciones de variables y más.

¡Nos vemos en el próximo capítulo!

Resumen del capítulo

- Cuando usas TypeScript, te recomendamos que también uses *TypeScript ESLint*.
- TypeScript ESLint es una herramienta de análisis de código estático que te ayuda a encontrar y corregir problemas comunes de calidad de código en código TypeScript.
- El flag `strict` del compilador de TypeScript nos ayuda a deshabilitar el uso implícito de `any` y el plugin de TypeScript ESLint nos ayuda a deshabilitar el uso explícito de `any`.
- El comentario `@ts-ignore` deshabilita la verificación de tipos de TypeScript para la siguiente línea.
- El comentario `@ts-nocheck` deshabilita la verificación de tipos de TypeScript para todo el archivo, siempre y cuando se use en la parte superior del archivo.
- El comentario `@ts-expect-error` te permite deshabilitar la verificación de tipos de TypeScript para la siguiente línea, solo si ya está fallando.
- La regla `@typescript-eslint/ban-ts-comment` deshabilita los comentarios de directiva de TypeScript, obligándote a corregir el error de tipo subyacente.
- La regla `@typescript-eslint/ban-types` obliga a usar `: string` en lugar de `: String`, `: number` en lugar de `: Number`, `: boolean` en lugar de `: Boolean`, etc.
- De ahora en adelante, todas las reglas de TypeScript ESLint estarán contenidas en una única prueba llamada `@typescript-eslint`.

Unidad 4: Primitive types

Se estudiaron los tipos primitivos de TypeScript, incluyendo `string`, `number`, `boolean`, `null` y `undefined`. También se aprendió a declarar variables con tipos explícitos y a aprovechar la inferencia automática de tipos.

Resumen del capítulo

Last updated: April 2024

¡Buen trabajo!

Volviendo al código del primer desafío:

```
function getFinalAmount(price: number, applyDiscount: boolean) {
  if (!applyDiscount) {
    return price;
  }

  const amount = price - (price * 10 / 100);
  return amount;
}
```

TypeScript a quite ayuda con una útil experiencia de autocompletado cuando escribes `getFinalAmount`, ya que obtendrás un autocompletado útil que te mostrará los argumentos que debes proporcionar:

```
getFinalAmount(price: number, applyDiscount:
boolean): number
getFinalAmount()
```

Además, previene errores si accidentalmente proporcionas tipos de argumentos incorrectos, por ejemplo:

```
getFinalAmount(50, "25%");
El argumento de tipo 'string' no es assignable al parámetro de
tipo 'boolean'.
```

En el próximo capítulo, aprenderemos sobre los tipos de unión. Esto finalmente nos permitirá trabajar con ejemplos más complejos y útiles. ¡Nos vemos allí!

Resumen del capítulo

- JavaScript tiene siete tipos primitivos: `string`, `number`, `boolean`, `undefined`, `null`, `bigint`, y `symbol`.
- TypeScript proporciona un tipo para cada uno de esos tipos primitivos.
- Puedes anotar las declaraciones de variables usando la misma sintaxis a la que estás acostumbrado. La anotación de tipo debe ir antes del signo de asignación (`=`), por ejemplo, `const username: string = "Alex";`
- TypeScript a menudo puede inferir el tipo de las declaraciones de variables observando el valor asignado. En esos casos, no tienes que escribir el tipo, ya que puede deducirse automáticamente.

Unidad 5: Union types

Se revisaron los tipos de unión utilizando el operador ``|``, permitiendo que una variable pueda aceptar más de un tipo de dato. También se aprendió a utilizar ``typeof`` para diferenciar comportamientos según el tipo recibido.

Resumen del capítulo

last updated February 2024

¡Gran trabajo!

Recuerda que todos los tipos de TypeScript que estás escribiendo desaparecerán de tu código. Solo son útiles en tiempo de compilación para prevenir errores de ejecución con tu código JavaScript.

Puedes comprobarla una vez más usando el [TS Playground oficial](#).

Intenta pegar el siguiente código:

```
function formatValue(value: string | number) {  
  if (typeof value === 'string') {  
    return value.toLowerCase();  
  } else {  
    return value.toFixed(2);  
  }  
}
```

Observa cómo `string | number` desaparece del código de salida. Por otra lado, `typeof` es un operador de JavaScript que se ejecuta en tiempo de ejecución, por lo tanto, permanece en el código de salida.

Si nos alejamos un poco, TypeScript ya nos está ayudando a detectar algunos errores en tiempo de ejecución al asegurarnos de que solo llamemos a métodos que están disponibles según el tipo de la variable. También nos beneficiamos de una rica experiencia de autocompletado en nuestro editor.

Sin TypeScript, es posible que hayas encontrado mensajes de error como:

```
No se puede llamar a .toLowerCase() en count (cuando count hubiera sido  
un número).
```

Como costarían estos beneficios, tenemos que familiarizarnos con TypeScript, su sintaxis y sus mensajes de error. ¡Pero vale totalmente la pena!

En el próximo capítulo, aprenderemos más sobre conceptos relacionados con funciones en TypeScript. ¡Nos vemos allí!

Resumen del capítulo

- Un **tipo de unión** es un nuevo tipo que creamos especificando que cualquiera de los dos (o más) tipos especificados puede ser aceptado.
- `string | number` es un tipo de unión entre `string` y `number`. Significa que el tipo puede ser un `string` o un `number`.
- Cuando tienes un tipo de unión, solo puedes llamar a un método si es válido en cada tipo proporcionado en el tipo de unión.
- El **estrechamiento** (narrowing) es cuando TypeScript puede deducir un tipo más específico basándose en tu código.
- Los tipos de TypeScript no están disponibles en tiempo de ejecución. Sin embargo, puedes usar el operador `typeof` de JavaScript para crear diferentes comportamientos basados en diferentes tipos en tiempo de ejecución.

Unidad 6: The type alias

Se aprendió a crear alias de tipos mediante la palabra clave `type`, facilitando la reutilización de definiciones de tipos y mejorando la organización y legibilidad del código.

Resumen del capítulo

last updated January 2026

¡Buen trabajo!

En el siguiente capítulo, aprenderemos sobre los tipos de arreglos. ¡Nos vemos allí!

Sobre el instructor

Una introducción tardía: Hola 🙌, soy [Jed Jaubren](#). Soy el escritor de este curso, creador y mantenedor de la plataforma donde estás estudiando.



Hablando en la cumbre de Google Developer Experts sobre las lecciones que aprendí al enseñar programación.

Si te interesan mis publicaciones de frontend, sígueme en [X](#). Si eres fan de los gatos o el snowboard, puedes seguirme en [Instagram](#). También puedes suscribirte a mi [canal de YouTube de tecnología](#).

⚠️ Ten en cuenta que estoy feliz de ofrecerte soporte a través de este sitio web (Centro de ayuda en la esquina inferior derecha de la página), pero desafortunadamente, no a través de plataformas sociales como X/Instagram.

Resumen del capítulo

- Puedes crear un nombre personalizado para un tipo usando un alias de tipo.
- Aquí tienes un ejemplo de un alias de tipo: `type StringOrNumber = string | number;`
- Los alias de tipo no se pueden cambiar.